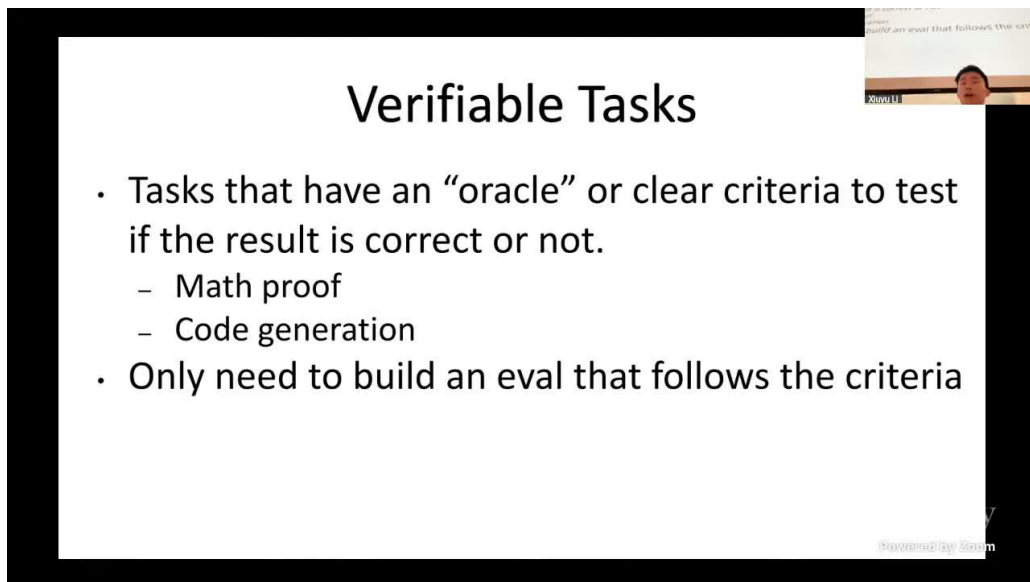


课程笔记

UCB CS294/194-196 Agentic AI (Fall 2025) 第 05 讲: Agent Evaluation & Project Overview

Codex

2026 年 4 月 16 日



The image is a screenshot of a video lecture slide. The slide has a black border and a white background. At the top center, the title 'Verifiable Tasks' is written in a large, bold, black font. Below the title, there is a bulleted list of two main points. The first point is 'Tasks that have an "oracle" or clear criteria to test if the result is correct or not.', followed by two sub-points: 'Math proof' and 'Code generation'. The second main point is 'Only need to build an eval that follows the criteria'. In the top right corner of the slide, there is a small video feed showing a person's face. At the bottom right of the slide, there is a small logo that says 'Powered by Zoom'.

Verifiable Tasks

- Tasks that have an “oracle” or clear criteria to test if the result is correct or not.
 - Math proof
 - Code generation
- Only need to build an eval that follows the criteria

Powered by Zoom

视频作者/频道: Berkeley RDI / Agentic AI MOOC

发布日期: 2025-10-06

视频时长: 1:34:22

视频链接: <https://www.youtube.com/watch?v=Vf0A2a0dj4w>

目录

1	为什么 agent 评估是核心问题	2
1.1	评估不是附属品	2
1.2	什么时候必须做评估	2
1.3	本章小结	2
2	评估的类型	2
2.1	close-ended, open-ended, verifiable, non-verifiable	2
2.2	人类评估与 LLM-as-a-Judge	3
2.3	静态评估与动态评估	3
2.4	本章小结	3
3	什么是好的 eval system	3
3.1	Outcome validity	3
3.2	评估最常见的失败方式	4
3.3	本章小结	4
4	案例研究：好 benchmark 是怎么做出来的	4
4.1	CyberGym, τ -bench, τ^2 -bench	4
4.2	GDPval, CRMArena, LegalAgentBench	4
4.3	本章小结	5
5	课程项目：green agent 应该怎么做	5
5.1	green agent 与 white agent	5
5.2	两类项目	5
5.3	Step-by-step checklist	5
5.4	Project grading rubric	6
5.5	本章小结	6
6	总结与延伸	6

1 为什么 agent 评估是核心问题

1.1 评估不是附属品

这一讲的第一句定义非常直接：evaluation 是对模型和 agent 的 *systematic, repeatable measurement*。对于 agent 来说，评估的重要性比普通 LLM 更高，因为 agent 不仅要回答问题，还要在动态环境里规划、调用工具、维持记忆，并完成多步任务。没有可信评估，训练进步、部署安全和 benchmark 竞争都无从谈起。

维度	LLM	Agent
状态	静态文本输入输出	动态环境 + 工具 + 历史交互
能力	生成、补全、总结	规划、执行、迭代、修复
评估难点	文本质量与事实性	任务成功率、路径可靠性、工具使用、环境状态变化
风险	幻觉、风格偏差	错误行动、状态污染、prompt/benchmark gaming

表 1: Agent eval 相比普通 LLM eval 的复杂度提升。

1.2 什么时候必须做评估

讲者给出的答案是：几乎在训练和上线的每个阶段都需要评估。训练前要评估数据质量和任务定义；训练中要看 loss、reward、pass rate 和中间行为；训练后要用 benchmark 和 harness 比较模型；上线前要看安全性、成本和用户体验；上线后还要持续监控漂移。对 agent 而言，评估不是最后一步，而是整个流水线里的闭环控制信号。

评价的三个用途

1. **比较模型**：公平比较不同模型和不同策略。
2. **指导决策**：决定是否可以部署、是否需要回滚。
3. **推动进步**：让 benchmark 本身成为研究对象，而不是静态分数板。

1.3 本章小结

这一节把评估的地位讲得很清楚：agentic 系统的每一次能力提升，最后都要通过可复现的评估来被确认。没有 evaluation，后续的训练、部署和 project grading 都失去参照物。

2 评估的类型

2.1 close-ended, open-ended, verifiable, non-verifiable

讲座把 agent 评估的第一层分类拆成几个很实用的维度。close-ended 任务的正确答案数量有限，因此容易自动化评估；open-ended 任务则更接近真实应用，但又必须进一步区分 *verifiable* 与 *non-verifiable*。前者像数学证明和代码生成，后者像故事创作、风格迁移和开放式写作。

类别	典型任务	评价方式	代表问题
Close-ended	多选、固定答案题	自动判分	答案空间是否有限?
Open-ended / verifiable	数学、代码	oracle、测试、checker	是否存在明确正确性标准?
Open-ended / non-verifiable	写作、风格、摘要	人类评估、LLM judge	结果是否高度主观?
Static	固定测试集	可复现对比	会不会很快被过拟合或污染?
Dynamic	持续更新测试集	持续追踪	能否保持相关性与抗污染?

表 2: 讲座中的评估分类。

2.2 人类评估与 LLM-as-a-Judge

人类评估通常被视为 gold standard，但它慢、贵、不可重复，而且不同标注者之间可能存在 disagreement，甚至同一标注者在不同时刻也会给出不同判断。LLM-as-a-judge 则提高了吞吐，但会遇到随机性、偏置、分数难解释、对 prompt 很敏感等问题。讲者的建议不是简单放弃，而是把它们当作可用但不完美的信号：要和 human eval 交叉验证，要做多次重复打分，要尽量用更细的 rubric 和连续分数。

2.3 静态评估与动态评估

静态 benchmark 的优点是稳定、可复现，适合学术比较；动态 benchmark 则更接近真实世界数据分布，也更不容易被污染或过拟合。举例来说，ImageNet、GLUE、MMLU 属于典型静态 benchmark，而 DynaBench、LiveCodeBench 则更强调动态更新。对于 agent 来说，动态环境几乎是必需的，因为工具、网页、API 和用户需求本来就在变化。

2.4 本章小结

这一部分给了评估的基础 taxonomy。最重要的不是名词，而是判断：当任务有明确 oracle 时，应尽量用 verifiable eval；当任务难以自动判分时，必须小心设计 human/LLM rubric，并警惕静态 benchmark 很快被污染或饱和。

3 什么是好的 eval system

3.1 Outcome validity

讲座用一个关键词概括“好评估系统”应该追求的东西：outcome validity。它的意思不是看模型是不是在某个 benchmark 上拿了高分，而是看这个分数是否真的对应了我们关心的真实任务结果。比如，在文本、代码、环境状态变化和多步推理上，判分信号都应尽量对应真正的 task success。

Outcome validity 的直觉

一个好 eval 不应该只测“模型会不会讨巧”，而应该测“模型完成真实任务的能力是否真的更强了”。如果评估可以被 shortcut、gaming 或数据污染轻易击穿，那么它就失去了定义 intelligence 的资格。

3.2 评估最常见的失败方式

讲者专门列出了几个常见坑：数据噪声和偏置会扭曲结论；评估如果不切实际，就无法反映 practitioner 的真实需求；如果 benchmark 可以被 shortcut，就会鼓励投机；如果任务不够难，就会很快饱和；如果测试集过于静态，公开之后就会污染得很快。简言之，评估既要可复现，又要难作弊，还要足够贴近真实世界。

一个很实用的判断标准

如果你发现模型在 benchmark 上不断变强，但真实任务效果没有同步提升，那就该怀疑 evaluation 本身是否已经失真。

3.3 本章小结

好的 eval system 不是“能算分”这么简单，而是要有 outcome validity。它要能解释真实任务成功、支持鲁棒比较，并且尽量抵抗污染、捷径和过拟合。

4 案例研究：好 benchmark 是怎么做出来的

4.1 CyberGym, τ -bench, τ^2 -bench

这一讲用多个案例说明如何把一个 benchmark 从“看起来像任务”做成真正可用的评估系统。CyberGym 的数据来自 ARVO 和 OSS-Fuzz 等真实世界漏洞来源，通过重建 pre/post patch commits、可执行文件和 ground-truth PoC，再用 runtime sanitizers 做自动判定。它的关键点是：不靠主观评分，而是通过是否真正触发漏洞来判定成功。

τ -bench 则把重点放在 tool-agent-user interaction 上。任务通常是多轮对话式的，agent 要在遵守 policy 的前提下，调用 API 工具完成用户目标，最后通过数据库状态与目标状态是否一致来判断成败。它天然支持 pass@1 与 pass@k 这类可靠性指标。

τ^2 -bench 在 τ -bench 的基础上进一步系统化，加入 LLM-drafted PRD、mock DB、unit tests，以及环境、通信、自然语言和 action 等多种 assert 分类。它想表达的核心是：benchmark 不只是题目集合，而是一套能自动验证的任务生成管线。

4.2 GDPval, CRMArena, LegalAgentBench

GDPval 的特点是把任务交给有丰富经验的专业人士来写，再由同领域的专家盲审 AI 与 human deliverables。它特别强调低 contamination 风险和真实工作流，适合衡量“像人一样完成工作”的能力。CRMArena 则更偏向 CRM 业务流程里的多步代理任务。LegalAgentBench 则把法律场景中的多跳检索、逻辑推理和法律写作整合到一个多工具环境里，工具数量和语料数量都相当大。

Benchmark	任务域	判分方式	关键设计点
CyberGym	cybersecurity	runtime sanitizer / crash trace	真实漏洞、pre/post patch、PoC
τ -bench	tool-agent-user	final DB state vs goal state	policy following、pass@k
τ^2 -bench	complex workflows	categorical assertions	PRD、mock DB、compositional tasks
GDPval	professional work	expert blind grading	低污染、真实 deliverable
LegalAgentBench	legal workflows	multi-tool success	17 corpora、37 tools、多跳推理

表 3: 讲座中用来说明“好 benchmark 应该长什么样”的案例。

4.3 本章小结

这些案例共同说明了一件事：好 benchmark 不是简单堆题，而是把 task、environment、metric、data pipeline 和 validation 一起设计好。越接近真实任务，越要重视 low contamination、difficulty stratification 和可自动验证的终点条件。

5 课程项目：green agent 应该怎么做

5.1 green agent 与 white agent

课程项目采用了一个很形象的分工：*green agent* 是评估方或主持方，*white agent* 是被测试的系统。*green agent* 的职责是 orchestrate 整个评估流程，读取任务、调用工具、收集结果、做判分，并把最终评估结果写回平台。它本质上就是一个评估技术员（technician），不是一个单纯跑分脚本。

5.2 两类项目

讲义把 project 分成两类。第一类是 *integrating existing benchmarks*：把已经发表、验证过的 benchmark 接到 AgentBeats / 课程平台里，再对其质量做分析、修正和扩展。第二类是 *building new benchmarks*：围绕一个新的任务域，自己设计环境、数据管线、评估指标和 green agent。

类型	主要工作	课程里特别看重什么
Integrate existing benchmark	适配已有 benchmark、分析质量问题、修正与扩展	Faithfulness、质量分析、可复用性
Build new benchmark	设计新任务、环境、metric、test cases、grader	Novelty、scope、validation、realism

表 4: 课程项目的两种主路线。

5.3 Step-by-step checklist

讲座给出的 checklist 非常实用：先选任务，再设计环境，再设计 metrics，最后设计 test cases。以订票 agent 为例，环境要定义浏览器或 APP、动作空间、环境反馈；metric 要定义成功率、价格、是否满足用户要求；test cases 要覆盖成功、失败、错误预订、找不到网站等多种情形。对 existing benchmark 来说，还要补上 manual validation、evaluator check、bias/limitation notes，并在必要时做 correction 和 expansion。

5.4 Project grading rubric

对新 benchmark, rubric 重点看 goal & novelty、scope & scale、evaluator quality、validation、reliability、quality assurance、realism 和 impact。对已有 benchmark, rubric 重点看 analysis、faithfulness、quality assurance、evaluator quality、validation、reliability 和 impact。换句话说，老师并不只看“你能不能把 benchmark 跑起来”，还看你是否理解它为什么成立、哪里有缺陷、你修复后是否更可靠。

5.5 本章小结

项目部分把整门课的评估思想落到可执行任务上：做 green agent 不是只做一个 wrapper，而是要完整设计任务、环境、metric、测试与判分逻辑。对于新 benchmark，重点是新能力空间；对于已有 benchmark，重点是质量分析和修复能力。

6 总结与延伸

这一讲把 agent eval 从“刷分”提升成了一门完整的方法论。最核心的结论是：评估必须和真实任务的 outcome 对齐，必须尽量可复现，并且必须能抵抗污染、捷径和饱和。对 agent 来说，评估不只是最后的验收，而是训练、部署和课程项目本身的一部分。

如果把这一讲和前后的内容连起来看，就会得到一个很清晰的 pipeline：先用可验证训练数据和奖励训练 agent，再用合适的 benchmark 与 harness 评估 agent，最后把评估机制本身也做成可以持续改进的系统。对于后续做课程项目的同学来说，真正的高分不在于“把模型跑出来”，而在于“把评估做对”。